2026 MITRE eCTF

University of Denver | Design Documentation

Advisors: Nate Evans (nathan.evans@du.edu) Adrian Rodriguez (adrian.rodriguez@du.edu)

Members: Jerome Althoff (jerome.althoff@du.edu), Jacob Fishbein (jacob.fishbein@du.edu), Domonic Velarde (domonic.velarde@du.edu), Warren Kankwe (warren.kankwe@du.edu), LaRon Latin (laron.latin@du.edu), Jason Velvick (jason.velvick@du.edu), Sy Pretto (lilia.pretto@du.edu)

Introduction: Secure Transfer of Circuit Fabrication Data

We are a team of 8 graduate students from the University of Denver dedicated to designing and implementing a secure transfer of circuit fabrication data via embedded hardware working through multiple Hardware Security Modules (HSMs). The HSMs are accessed by either the engineer or technician via the management interface. The engineer may need to access engineering data from the second HSM, but only the engineer's HSM should be able to do so. To restrict access, specific permission groups need to be implemented to where certain actions on certain files require specific groups. If prompted by the management interface, the transfer interface will handle the actual communication and transmission of files and file metadata from one HSM to the other.

Table of Contents

Section 1 -- What We Are Protecting Against 1

Section 2 – How The System Works : 2

Section 3 – Cryptography Overview: 4

Section 4 – Meeting The Security Requirements: 6

Section 5 – Ancillary Information: 7

Section 1 -- What We Are Protecting Against :

Our Embedded system involves two separate HSM devices that run on two Texas Instruments MSP-LITO-L2228 boards. The data will be transferred from the customer site, where their HSM equipped board will connect to the Technician's HSM equipped board for data transfer. As the initial design stands, this data is transferred unencrypted as plaintext between the boards. This allows the possibility for any attacker that can connect to the system with the ability to intercept the transfer and obtain the data. This means one of the main vulnerabilities we need to address is that of Man in The Middle (MITM) attacks. One of the main methods we will use to address this vulnerability is the use of cryptographic algorithms and secure secret storage (detailed in section 3) to ensure data in transit is confidential.

Some threats which must be considered: brute forcing the PIN (1), downgrading security configuration (2), side channel attacks (3), and arbitrary code execution (4). Considering these in ascending order of severity:

(1) The fastest PIN protected command has a timing requirement of 500 milliseconds. With 1 million possible PIN values, we should expect our PIN to be guessed within 500,000 attempts. At 2 guesses per second, it will take an attacker around 3 days to determine our PIN. If we find a way to persist a timer across power cycles to impose the permitted 5 second penalty, then the attack would take 10 times longer. Once compromised, our attacker will be able to use the PIN to perform the commands which are already allowed by the HSM security configuration.

(2) An attacker who can modify memory locations will be able to downgrade our security configuration. Without modifying our security logic, an attacker who reads our documentation and understands our data structure will be able to add arbitrary permissions to interact with our filesystem and the neighboring HSM's filesystem (read only).

(3) If it is possible to infer AES keys via passive monitoring of power consumption or RF emissions, or via other side channels, attackers will also gain full read of both local and neighboring HSMs, as well as write for the local HSM.

(4) Finally, if an attacker can exploit a software vulnerability to gain arbitrary code execution, they will be able to brute force our PIN much more quickly. Even if we are using secure hardware elements, any software-imposed rate limits can be easily bypassed, as well as any other cryptographic operations like signature verification. If this can be exploited over UART, then attackers will gain nearly unlimited control over our system.

Section 2 – How The System Works :

The critical path of our system consists of two HSMs needing to securely transfer sensitive data to and from one another. Specifically, there are six distinct commands that the HSMs can run based on the needed data transfer.

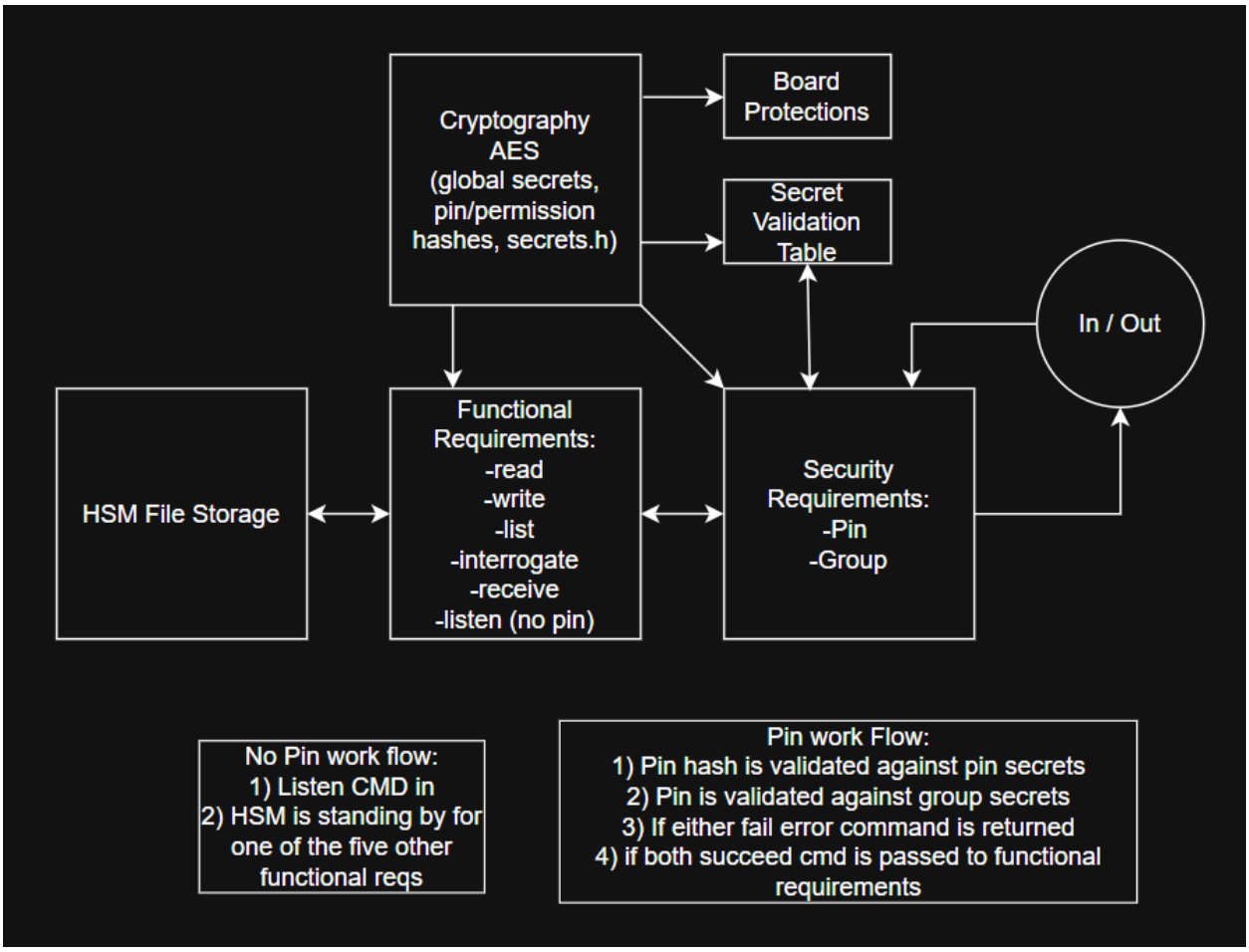
Communication between HSMs will be secured via an AES channel, enciphered with a shared secret which is established at build time. Keys will be stored in the Secure Key Storage module, which, after our firmware is done loading, can only be accessed by the AESADV module. This module will be responsible for all encipherment, avoiding risky software implementation. We will depend on these two hardware security modules for mitigation against any side-channel attacks which might target our AES keys.

The security configuration will be stored in a read-only section of flash memory. This memory region will be configured by our firmware as read-only during the boot process. The security configuration will consist of the group permissions and a PIN hash.

Whenever a file is being transferred, it will first pass through the AESADV module, either to be sent to the management interface as plaintext or to be stored in flash memory as ciphertext. Files will never be stored as plaintext.

Our security logic will be stored in read-only flash memory. It will enforce the permissions and the PIN while also executing the commands, including calling the AESADV module as needed.

The above constitutes our Minimum Viable Product. As time and budget allow, we will make use of additional hardware features to reduce our attack surface. We will also develop a more robust cryptosystem to deter any bypassing of our security logic in the event an attacker is able to gain arbitrary code execution and call our AESADV module directly. These approaches are detailed in Sections 4 and 3, respectively.



Section 3 – Cryptography Overview:

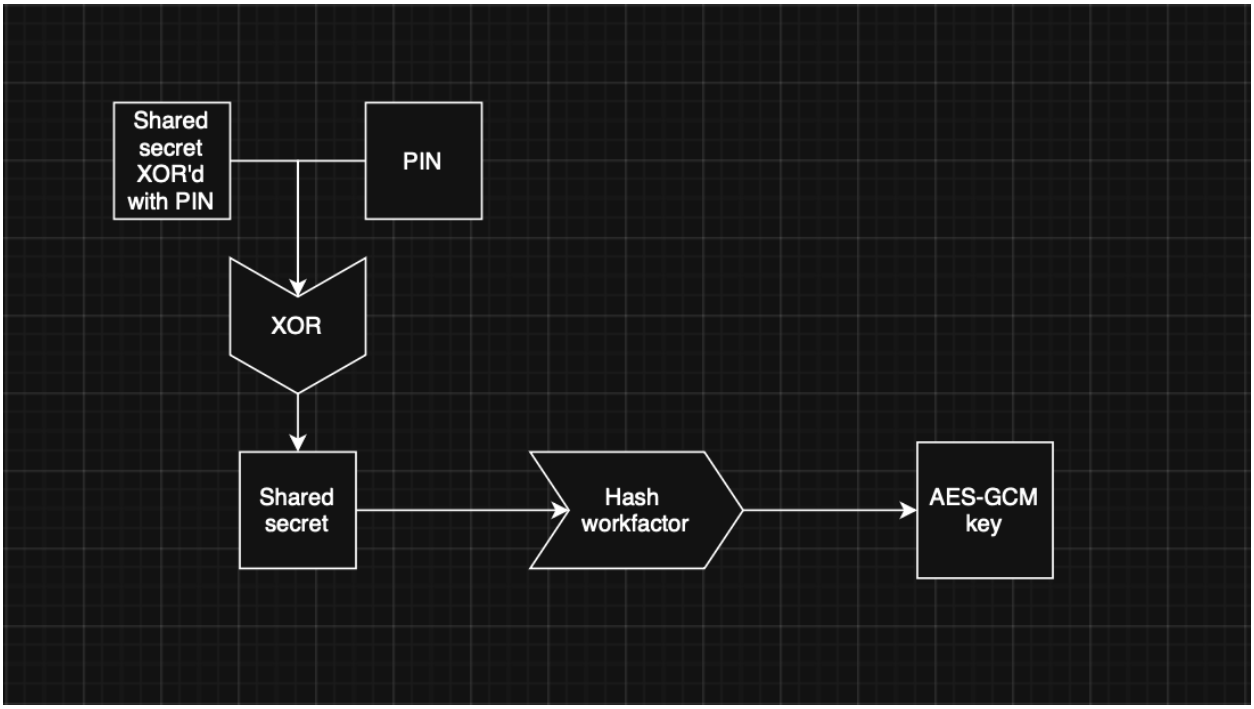
The initial implementation will use a symmetric AES-GCM shared-key system to establish a secure communication channel between the two HSMs. The design will support potential future integration of Ed25519-based certificates to provide cryptographic identity verification and authorization during the initial handshake phase. This system would allow the sending board to verify the receiving board is actually who it says it is, before establishing an encrypted connection over UART.

As stated before, the main cryptographic system for confidential data transfer will be through symmetric AES encryption. This symmetric encryption system will begin with an initial handshake, where the two HSMs will prove identity via pre shared secrets before any data transfer begins. Any board running our system will not begin to transfer any data unless it is able to authenticate the other HSM. For the system to reference which user has what permissions, such as read, write, list, etc, we will use a secure access control list (ACL) to store specific user permissions and actions.

We will implement a key derivation function which combines the shared secret and PIN as inputs, then passes the result through a work factor, the result of which will be our AES-GCM key used for encryption and decryption of files and communications (this AES mode also contains a MAC, useful for authentication and integrity checks).

Because HSMs will have different PINs, we will XOR the shared secret and PIN before storage. This will require knowledge of the PIN to recover the shared secret (by XOR-ing with the PIN again). Because the shared secret is then used as an input to our KDF, this means that our attacker is cryptographically forced to run the work factor for every PIN guess attempt. The work factor will be tuned to meet the functional timing requirements.

This will slow down attackers who are able to gain unauthorized access to the memory regions containing our secret. Even if an attacker copies a secret to another machine for faster offline cracking of our PIN, it will take them longer than if we had simply stored a PIN hash. This will also buy time against attackers who are able to gain remote code execution and thus are able to bypass any rate limits, but who do not yet know our PIN. The key derivation process is pictured below.



Section 4 – Meeting The Security Requirements:

There are three main security requirements that our secure design seeks to address. The main requirements for security are as follows:

Security Requirements (SR)	Description	Solution
SR1	An attacker should not be able to perform any file action without a validly provisioned HSM with the permissions to perform that action on files belonging to that group.	Authentication via initial AES symmetric handshake before any data transfer begins
SR2	No PIN-protected action should be able to be completed by a user without prior knowledge of the PIN.	Valid PINs will be stored securely via hashing with a shared secret
SR3	An HSM should not successfully receive any file that was not generated by another valid HSM with write permissions for that group.	Data transfer will only occur over encrypted AES tunnel after an initial handshake providing mutual authentication

There are two additional hardware modules (not explicitly security related) which are potentially useful to us. They are designed for low power operation and provide potential paths to bypass the main CPU and memory. These paths allow us to continue addressing some of the other identified risks, such as arbitrary code execution and memory modification.

The DMA system is designed to directly read and write between memory and peripherals. This is useful in low power environments where peripherals need to collect data continuously. Meanwhile, the CPU can be woken up only as needed for occasional batch processing. This might be useful for collecting instrument data and publishing it to a central database every 24 hours, for example.

DMA is tightly integrated into the system, meaning that it can generate and respond to events as well as directly interact with other modules. For example: DMA can respond to incoming UART data and send it to the AESADV module, which can then send it back to DMA to be copied into flash memory, or back out over UART, or to the management interface. This could theoretically allow for a complete bypass of the CPU for crypto-enabled read and write operations, even running during a low power or sleep mode.

Finally, we will investigate the VBAT island. This is an isolated power domain in the system which runs off of a small battery. Its main purpose is to keep time accurately, so the clock speed of this island is 1,000 times slower than the CPU (plenty accurate for timekeeping). There are two additional features of this module which are relevant to our security model: tamper detection, and scratchpad memory. The memory is very limited, only 256 bytes. If we are able to encode an efficient binary representation of our security configuration, we can store it on the VBAT island and use the clock to enforce rate limits which can survive HSM power loss. We can also require a power cycle any time the tamper detection timestamp is more recent than when the HSM was booted, communicated by an error message sent to the host.

Utilizing both of these approaches, the role of the CPU during normal operation is greatly reduced. The main purpose of the central processor is then to initialize the hardware modules which will go on to do much of the processing independently. Afterwards, it calls INITDONE and disappears from the scene (except perhaps for a few interrupt vectors for DMA to call every now and then).

Section 5 – Ancillary Information:

Texas Instruments MSPM0L2228 User Guide and Datasheet were referenced when identifying which modules on the board could be relevant to the design. Some key modules from these documents we reviewed include the VBAT island (User Guide Page 1503), AESADV module (User Guide Page 728), and the DMA system (Page 430).

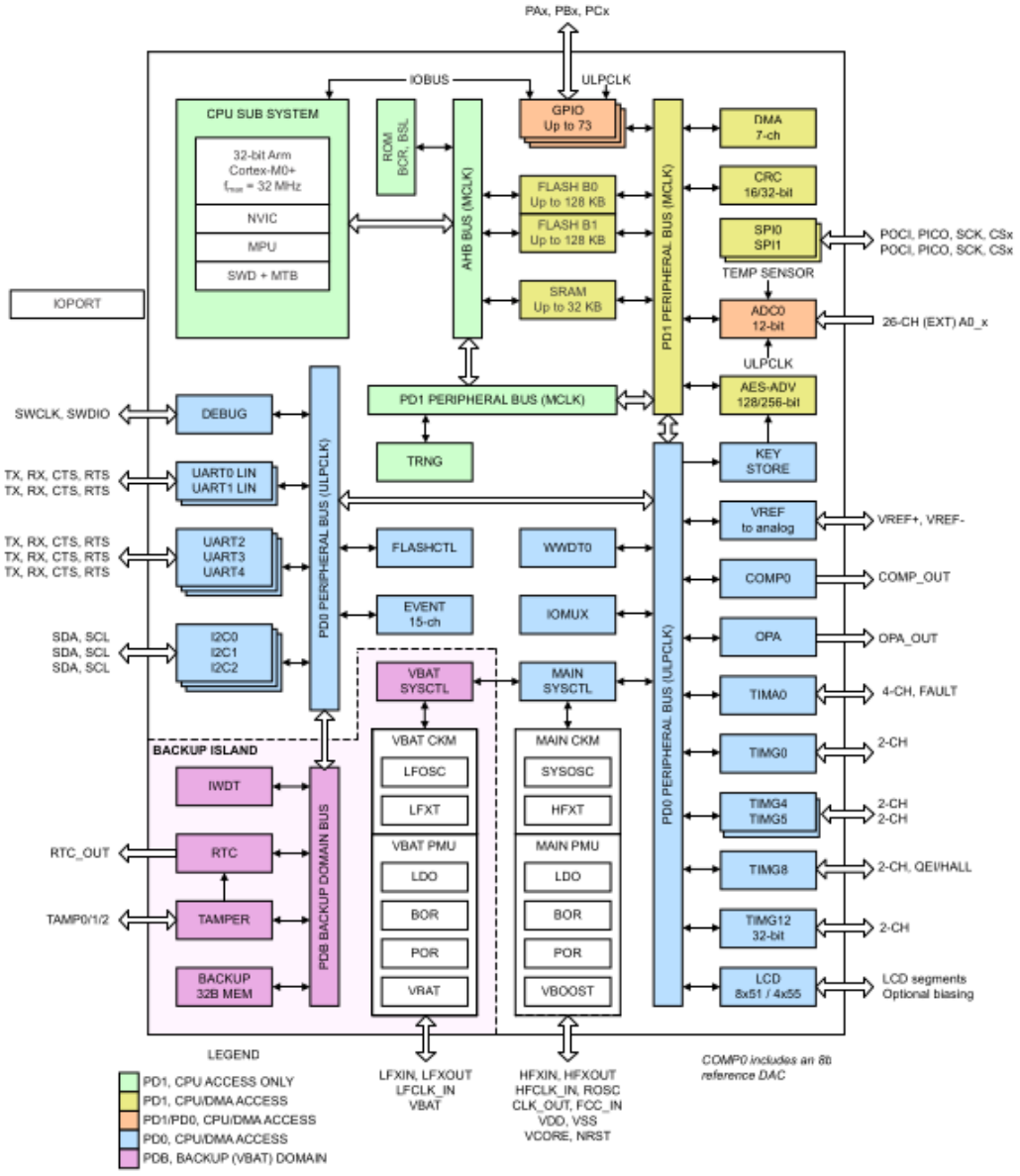


Figure 1-1. MSPM0Lxx Bus Organization

Figure Shows MSPM0L2228 Board Architecture (User Guide Page 14)